

Quiz 2: Persistencia Avanzada, Hibernate Internals, Manejo de Errores y Seguridad JWT

Curso: IF0009 - Desarrollo de Software IV

Docente: Mag. Jonathan Granados C.

Puntuación Total: 25 Puntos

Tiempo Límite Recomendado: 25 - 30 minutos

Instrucciones: Lea detenidamente cada uno de los siguientes 5 casos prácticos y ejercicios de rastreo de código. Seleccione la opción de respuesta que considere correcta. Al finalizar la prueba, puede consultar el solucionario analítico ubicado al final del documento para verificar sus razonamientos técnicos.

■ Cuestionario de Evaluación Práctica (< 30 Minutos)

1. Ciclo de Vida de Entidades JPA/Hibernate y Dirty Checking (5 Puntos)

Un desarrollador ejecuta un método anotado con `@Transactional` en la capa de servicio para actualizar el precio de un producto en el sistema. Dentro del método, busca la entidad por ID mediante `productoRepository.findById(id)`, ejecuta la modificación mediante el método modificador `producto.setPrecio(nuevoPrecio)` y **NO** invoca explícitamente el método `productoRepository.save(producto)`. Sin embargo, al concluir la ejecución de la transacción, la base de datos ejecuta automáticamente la instrucción SQL `UPDATE`.

¿Cuál es el mecanismo interno de Hibernate que produce esta sincronización implícita y en qué estado se encuentra la entidad durante la transacción?

- ☐ A) La entidad se encuentra en estado *Transient* y Spring Data JPA dispara un evento asíncrono que recrea la tabla en la base de datos mediante sentencias DDL.
- ☐ B) La entidad se encuentra en estado *Persistent* (asociada al Contexto de Persistencia), por lo que Hibernate realiza *Dirty Checking* al ejecutar el *flush* automático y sincroniza los cambios en la base de datos sin requerir `save()`.

- [] C) La entidad se encuentra en estado *Detached* y el repositorio activa una captura de excepciones de base de datos que reintenta la modificación mediante disparadores (*triggers*) SQL.
- [] D) El comportamiento se debe a un error de compilación de Hibernate; las entidades en JPA nunca pueden sincronizarse con la base de datos a menos que se anote la clase con `@DynamicUpdate`.

2. Diagnóstico de Rendimiento ORM: El Problema N+1 SELECT (5 Puntos)

En una API REST de logística y envíos, un endpoint `@GetMapping("/api/v1/empresas")` retorna una lista de 50 empresas de transporte. Cada entidad `EmpresaLogistica` mantiene una relación `@OneToMany` mapeada hacia su colección de `vehiculo` con la estrategia por defecto (`FetchType.LAZY`). Al consultar la lista completa de empresas junto con sus vehículos asociados para la respuesta JSON, las herramientas de perfilado muestran que Hibernate ejecuta **51 consultas SQL SELECT independientes** para atender una sola petición HTTP.

¿Cómo se denomina esta falla de rendimiento en mapeadores relacionales y cuál es la solución técnica más eficiente mediante Spring Data JPA?

- [] A) Se denomina *Heap Overflow Failure* y se soluciona cambiando la anotación de la relación a `@OneToOne(fetch = FetchType.EAGER)`.
- [] B) Se denomina *Problema N+1 SELECT* y se soluciona definiendo una consulta JPQL con `JOIN FETCH` (ej. `@Query("SELECT e FROM EmpresaLogistica e JOIN FETCH e.vehiculos")`) para recuperar las empresas y sus colecciones en una única consulta SQL.
- [] C) Se denomina *Deadlock de Persistencia* y se resuelve eliminando la anotación `@Entity` e instanciando un controlador nativo JDBC con cursores en memoria.
- [] D) Se denomina *Circular Dependency Defect* y se remedia agregando la directiva `@JsonIgnoreProperties` en la interfaz del repositorio de datos.

3. Ejercicio de Rastreo de Código: Intercepción Global de Errores (@RestControllerAdvice) (5 Puntos)

Analice la siguiente implementación de un manejador centralizado de excepciones en Spring Boot:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(RecursoNoEncontradoException.class)
    public ResponseEntity<Map<String, Object>> manejarNoEncontrado(
        RecursoNoEncontradoException ex) {

        Map<String, Object> error = new HashMap<>();
        error.put("timestamp", LocalDateTime.now());
        error.put("status", 404);
        error.put("error", "Not Found");
        error.put("message", ex.getMessage());

        return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error);
    }
}
```

Suponga que un cliente realiza una consulta HTTP a un endpoint inexistente y el servicio lanza `throw new RecursoNoEncontradoException("El vehículo con placa ABC-123 no existe")`.

¿Qué inconsistencia de protocolo ocurre en la respuesta entregada al cliente HTTP al ejecutar este código?

- ☐ A) El cliente recibe una cabecera con código de estado HTTP 400 Bad Request, a pesar de que el cuerpo JSON interno indica "status": 404, debido a que `ResponseEntity.status(...)` fija el código HTTP final del paquete.
- ☐ B) El cliente recibe un código HTTP status 500 Internal Server Error porque las excepciones personalizadas no permiten el retorno de colecciones Map.
- ☐ C) El cliente recibe una respuesta exitosa HTTP status 200 OK con un cuerpo vacío debido a la falta de la anotación `@ResponseBody`.
- ☐ D) El código genera una excepción de compilación `TypeMismatchException` en Spring Boot porque `@ExceptionHandler` solo admite tipos primitivos.

4. Ejercicio de Rastreo de Código: Evaluación de Control de Acceso (Spring Security & RBAC) (5 Puntos)

Examine la siguiente configuración de la cadena de filtros de seguridad en un sistema Spring Boot 3:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/v1/auth/**").permitAll()
            .requestMatchers(HttpMethod.GET, "/api/v1/envios/**")
                .hasAnyRole("CONDUCTOR", "ADMIN")
            .requestMatchers(HttpMethod.POST, "/api/v1/envios/**")
                .hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .sessionManagement(sess -> sess
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS)
        );
    return http.build();
}
```

Un usuario autenticado cuyos datos en el token JWT otorgan la autoridad `ROLE_CONDUCTOR` intenta enviar la siguiente solicitud HTTP a la API REST:

POST `/api/v1/envios/` con el cuerpo JSON de un nuevo paquete.

¿Cuál es el resultado exacto generado por el módulo de seguridad ante esta solicitud?

- ☐ A) La solicitud se procesa con éxito retornando `201 Created`, ya que la directiva `permitAll()` al inicio del bloque anula los controles posteriores.
- ☐ B) La capa de seguridad deniega el acceso y retorna el código de estado HTTP `status 403 Forbidden`, debido a que el rol `ROLE_CONDUCTOR` no cuenta con la autoridad `ROLE_ADMIN` exigida para el verbo `POST` en esa ruta.
- ☐ C) La solicitud redirige automáticamente al usuario a una página HTML de inicio de sesión con el código HTTP `302 Found`.
- ☐ D) Spring Security genera una excepción runtime `ExpiredJwtException` y reinicia el contenedor embebido Tomcat.

5. Seguridad en Sesiones JWT: Anatomía y Mecanismo de Revocado (RFC 7519) (5 Puntos)

Una empresa implementa autenticación mediante JSON Web Tokens (JWT) con arquitectura *stateless*. Durante una prueba de penetración, el equipo de ciberseguridad descubre que si un cliente realiza un cierre de sesión (*logout*) en la interfaz web eliminando el token de su almacenamiento local (`localStorage`), una cadena JWT interceptada previamente por un tercero continúa siendo aceptada como válida por los endpoints del back-end hasta que se cumpla su tiempo de expiración (`exp`).

¿A qué característica del estándar JWT se debe este comportamiento y cuál es la solución técnica recomendada para permitir la invalidación inmediata de tokens?

- ☐ A) Ocurre porque los JWTs no soportan firma digital. Se soluciona cambiando la codificación de Base64URL a XML estático.
- ☐ B) Ocurre porque los JWTs son tokens autosuficientes de estado cliente (*stateless*) y el servidor no guarda sesión. Se resuelve implementando una lista de revocado (*Token Blacklist / Denylist*) que registre los identificadores `jwt` invalidados y los verifique en el filtro `JwtAuthenticationFilter`.
- ☐ C) Ocurre porque se omitió la anotación `@CrossOrigin` en el controlador de autenticación. Se soluciona activando las cookies de sesión tradicionales `JSESSIONID`.
- ☐ D) Ocurre porque la clave secreta HMAC-SHA256 ha caducado en el servidor. Se soluciona reiniciando la aplicación Spring Boot.

■ Solucionario Analítico y Justificaciones Pedagógicas

Pregunta 1: Ciclo de Vida de Entidades JPA/Hibernate y Dirty Checking

- Respuesta Correcta: B**
- Justificación Analítica:**
- Al consultar una entidad a través del repositorio dentro de un método `@Transactional`, dicha entidad ingresa al Contexto de Persistencia (Session de Hibernate) en estado **Persistent**.

Durante el ciclo de vida de la transacción, Hibernate guarda una copia previa (*snapshot*) de la entidad. Al momento del *flush* (previo al *commit* de la transacción), Hibernate ejecuta el algoritmo de **Dirty Checking** (detección de cambios), compara la entidad modificada contra su *snapshot* original e invalida el estado emitiendo automáticamente la sentencia SQL `UPDATE` requerida sin necesidad de llamar de manera manual a `save()`.

- **Análisis de Distractores:**
- **A es incorrecta:** El estado *Transient* corresponde a objetos instanciados con `new` que aún no han sido asociados al Contexto de Persistencia ni poseen identificador clave primaria.
- **C es incorrecta:** El estado *Detached* ocurre cuando la sesión de persistencia se cierra o la entidad se desvincula; las modificaciones en estado *Detached* no se sincronizan automáticamente a menos que se invoque explícitamente `merge()`.
- **D es incorrecta:** El comportamiento de *Dirty Checking* es una característica central estándar de JPA y Hibernate, no requiere `@DynamicUpdate` para operar (dicha anotación solo optimiza las columnas incluidas en la cláusula `SET`).

Pregunta 2: Diagnóstico de Rendimiento ORM: El Problema N+1 SELECT

- **Respuesta Correcta: B**
- **Justificación Analítica:**
- El **Problema N+1 SELECT** ocurre cuando una consulta inicial recupera `$N$` entidades principales (1 consulta para 50 empresas) y, al iterar o serializar la colección relacionada con la estrategia `FetchType.LAZY`, el ORM se ve obligado a disparar `$N$` consultas adicionales individuales (50 consultas extra para traer los vehículos de cada empresa), sumando un total de `$N+1$` (51) consultas a la base de datos. La solución técnica canónica en JPA/Hibernate es utilizar la cláusula **JOIN FETCH** en JPQL o un `@EntityGraph`, indicando al motor de persistencia que construya un `INNER JOIN` o `LEFT OUTER JOIN` SQL para cargar en un solo viaje (*single round-trip*) la entidad padre y sus colecciones asociadas.
- **Análisis de Distractores:**
- **A es incorrecta:** Modificar la relación a `@OneToOne` altera la cardinalidad del modelo de dominio y no soluciona el patrón de carga.
- **C es incorrecta:** No se trata de un bloqueo de concurrencia (*Deadlock*) ni exige abandonar JPA para usar JDBC manual.

- **D es incorrecta:** `@JsonIgnoreProperties` evita ciclos infinitos en la serialización Jackson, pero no resuelve las peticiones SQL secundarias generadas en la capa de datos.

Pregunta 3: Ejercicio de Rastreo de Código: Intercepción Global de Errores (@RestControllerAdvice)

- **Respuesta Correcta: A**
- **Justificación Analítica:**
 - En la construcción de la respuesta dentro del manejador de excepciones, el desarrollador estructuró el cuerpo JSON colocando la propiedad `"status": 404`. No obstante, en la instrucción de retorno invocó `ResponseEntity.status(HttpStatus.BAD_REQUEST).body(error)`. El método `status(...)` de `ResponseEntity` define la cabecera real de la respuesta HTTP del protocolo. Como resultado, el servidor responderá a la red con el código **HTTP status 400 Bad Request**, creando una discrepancia de protocolo contra el valor 404 escrito manualmente dentro de la carga JSON. Para corregir este defecto y alinearse a estándares como RFC 7807 (Problem Details), el estado del `ResponseEntity` debe ser `HttpStatus.NOT_FOUND`.
- **Análisis de Distractores:**
 - **B es incorrecta:** Spring Boot serializa correctamente cualquier objeto Java (incluyendo mapas `Map<String, Object>`) a JSON mediante la librería Jackson.
 - **C es incorrecta:** La anotación `@RestControllerAdvice` incluye sintácticamente `@ResponseBody` de forma implícita para todos sus métodos.
 - **D es incorrecta:** La anotación `@ExceptionHandler` acepta clases de excepción personalizadas que extiendan de `Throwable` o `Exception`.

Pregunta 4: Ejercicio de Rastreo de Código: Evaluación de Control de Acceso (Spring Security & RBAC)

- **Respuesta Correcta: B**
- **Justificación Analítica:**

- Spring Security evalúa las reglas declaradas en `authorizeHttpRequests` en orden secuencial. La regla `.requestMatchers(HttpMethod.POST, "/api/v1/envios/**").hasRole("ADMIN")` exige explícitamente que la petición con método POST posea la autoridad `ROLE_ADMIN`. Dado que el usuario únicamente posee el rol `ROLE_CONDUCTOR` (el cual solo le permitiría realizar peticiones GET según la regla previa), la capa de autorización rechaza la solicitud. Al tratarse de una arquitectura *stateless* REST (sin formularios de redirección web), Spring Security responde con el código estándar de acceso denegado **HTTP 403 Forbidden**.
- **Análisis de Distractores:**
- **A es incorrecta:** `permitAll()` sólo aplica para las rutas coincidentes con `/api/v1/auth/**`, no para las rutas de `/api/v1/envios/**`.
- **C es incorrecta:** En aplicaciones configuradas con `SessionCreationPolicy.STATELESS`, no existen sesiones de navegador ni redirecciones HTML 302 Login; se responde directamente con códigos de estado HTTP RESTful (401 o 403).
- **D es incorrecta:** Una falla de autorización no dispara excepciones de expiración de token ni reinicia los servicios del servidor web.

Pregunta 5: Seguridad en Sesiones JWT: Anatomía y Mecanismo de Revocado (RFC 7519)

- **Respuesta Correcta: B**
- **Justificación Analítica:**
- Por especificación (RFC 7519), los JSON Web Tokens son criptográficamente autosuficientes y **stateless**: contienen toda la información de identidad y su firma HMAC/RSA en la misma cadena, permitiendo que el servidor los valide verificando únicamente la firma sin consultar una base de datos de sesiones. Esto significa que borrar el token en el cliente no invalida la firma matemática del JWT en el servidor. Para lograr la revocación inmediata antes del tiempo de expiración (`exp`), se requiere guardar una lista negra (*Blacklist*) de los identificadores únicos de token (`jti` - JWT ID) en una memoria rápida (como Redis o tabla de base de datos) y verificar en el `JwtAuthenticationFilter` si el `jti` entrante ha sido marcado como revocado.
- **Análisis de Distractores:**

- **A es incorrecta:** Los JWTs firman su contenido digitalmente y pueden ser cifrados (JWE); la razón de su persistencia no es la codificación Base64URL sino su naturaleza autocontenida sin estado en el servidor.
- **C es incorrecta:** La anotación `@CrossOrigin` resuelve restricciones de política de mismo origen (CORS) en navegadores, no la invalidación o gestión del estado de tokens de seguridad.
- **D es incorrecta:** Reiniciar la aplicación no revoca firmas válidas mientras la clave secreta de firmado se mantenga idéntica en las propiedades del sistema.